Lab 4: Pose Goals & Obstacle Avoidance

Student

Name: □□

Date: 2026-04-27

Overview

This lab extends the SO101 MoveIt workflow from joint-space replay to end-effector pose goals. I implemented a new MoveToPose action, completed the pose action server, added a planning scene table collision object, added a TF-based pose saving service, and created a go_to_poses app that moves through saved end-effector poses while controlling the gripper separately.

The final robot demo used the go_to_poses app to lift the ArUco cube. The obstacle-avoidance portion was demonstrated separately in RViz by loading a table collision object and dragging the arm into the table so MoveIt marked the state as invalid/red.

Deliverable Checklist

- Deliverable 1: move_to_pose_server code included below.
- Deliverable 2: planning_scene code included below.
- Deliverable 3: RViz screenshot included below.
- Deliverable 4: save_pose.py code included below.
- Deliverable 5: go_to_poses.py code included below.
- Deliverable 6: Video link included below.
- Deliverable 7: FK vs IK paragraph section included for the required student-written paragraph.

Deliverable 1: MoveToPose Action And Server

Action Definition: MoveToPose.action

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_interfaces/action/MoveToPose.action
```

```
# Goal: target pose for the end effector
geometry_msgs/Pose target_pose
---
# Result
bool success
string message
---
# Feedback
float64 distance_to_goal
```

Interface Build Update: soa_interfaces/CMakeLists.txt

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_interfaces/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.10)

if(POLICY CMP0148)
  cmake_policy(SET CMP0148 OLD)
endif()

project(soa_interfaces)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Gripper.action"
  "action/MoveToJointStates.action"
  "action/MoveToPose.action"
  "srv/SwitchController.srv"
  "srv/SavePose.srv"
  "srv/SaveJointStates.srv"
  DEPENDENCIES geometry_msgs sensor_msgs
)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # comment the line when a copyright and license is added to all source files
  set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # comment the line when this package is in a git repo and when
  # a copyright and license is added to all source files
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

Pose Action Server: move_to_pose_server.py

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_functions/soa_functions/move_to_pose_server.py
```

This server receives an end-effector target pose, validates the target, asks MoveIt to plan a trajectory, executes the trajectory, and publishes feedback as distance-to-goal. Since the SO101 arm has only 5 DOF, it tries tight full-pose planning, relaxed-orientation planning, and then position-only planning.

```python
#!/usr/bin/env python3
"""
MoveToPose action server for the SOA 5-DOF arm.

Uses pymoveit2 to plan and execute IK-based motion to a target pose.
Implements a fallback strategy for the 5-DOF arm:
  1. Attempt full pose (position + orientation)
  2. Fall back to position-only IK if full pose planning fails

Usage:
    ros2 run soa_functions move_to_pose_server
"""

import math
import time

import rclpy
from rclpy.action import ActionServer
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from pymoveit2 import MoveIt2, MoveIt2State

from soa_interfaces.action import MoveToPose
from soa_functions import soa_robot


class MoveToPoseServer(Node):

    def __init__(self):
        super().__init__('move_to_pose_server')

        self.declare_parameter('max_velocity', 0.5)
        self.declare_parameter('max_acceleration', 0.5)
        self.declare_parameter('tolerance_position', 0.01)
        self.declare_parameter('tolerance_orientation', 0.1)
        self.declare_parameter('tolerance_orientation_relaxed', 0.5)
        self.declare_parameter('num_planning_attempts', 5)
        self.declare_parameter('allowed_planning_time', 3.0)
        self.declare_parameter('max_reach', 0.4)

        self._cb_group = ReentrantCallbackGroup()

        self._moveit2 = MoveIt2(
            node=self,
            joint_names=soa_robot.joint_names(),
            base_link_name=soa_robot.base_link_name(),
            end_effector_name=soa_robot.end_effector_name(),
            group_name=soa_robot.MOVE_GROUP_ARM,
            callback_group=self._cb_group,
        )

        self._moveit2.max_velocity = (
            self.get_parameter('max_velocity').get_parameter_value().double_value
        )
        self._moveit2.max_acceleration = (
            self.get_parameter('max_acceleration').get_parameter_value().double_value
        )
        self._moveit2.num_planning_attempts = (
            self.get_parameter('num_planning_attempts')
            .get_parameter_value().integer_value
        )
        self._moveit2.allowed_planning_time = (
            self.get_parameter('allowed_planning_time')
            .get_parameter_value().double_value
        )

        self._action_server = ActionServer(
            self,
            MoveToPose,
            'move_to_pose',
            self._execute_callback,
            callback_group=self._cb_group,
        )

        self.get_logger().info('MoveToPose action server ready')
```

Deliverable 2: Planning Scene

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_functions/soa_functions/planning_scene.py
```

This node adds the table as a MoveIt collision box so the planner rejects paths that intersect the table.

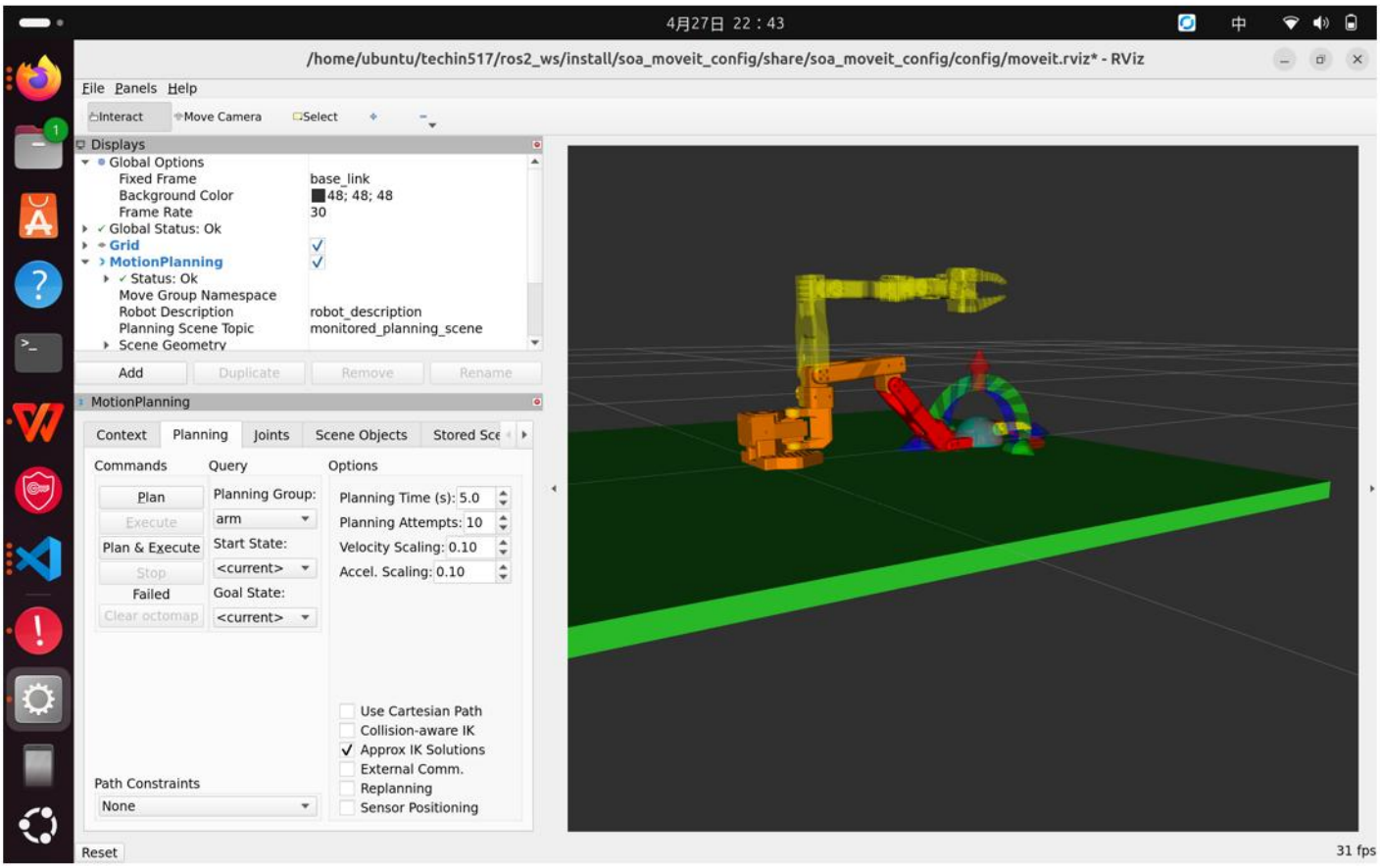
```
#!/usr/bin/env python3
"""
ROS 2 node that adds a table collision object to the MoveIt planning scene.

The table is a flat box at base_link level, representing the surface the robot
is mounted on. This prevents MoveIt from planning paths through the table.

Usage:
    ros2 run soa_functions planning_scene
    ros2 run soa_functions planning_scene --ros-args \
        -p table_position:="[0.0, 0.0, -0.01]" \
        -p table_size:="[1.0, 1.0, 0.02]"
"""

from threading import Thread

import rclpy
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.node import Node

from pymoveit2 import MoveIt2
from soa_functions import soa_robot


def main():
    rclpy.init()

    node = Node("planning_scene")

    node.declare_parameter("table_position", [0.0, 0.0, -0.01])
    node.declare_parameter("table_size", [1.0, 1.0, 0.02])

    callback_group = ReentrantCallbackGroup()

    moveit2 = MoveIt2(
        node=node,
        joint_names=soa_robot.joint_names(),
        base_link_name=soa_robot.base_link_name(),
        end_effector_name=soa_robot.end_effector_name(),
        group_name=soa_robot.MOVE_GROUP_ARM,
        callback_group=callback_group,
    )

    executor = rclpy.executors.MultiThreadedExecutor(2)
    executor.add_node(node)
    executor_thread = Thread(target=executor.spin, daemon=True, args=())
    executor_thread.start()
    node.create_rate(1.0).sleep()

    position = list(
        node.get_parameter("table_position").get_parameter_value().double_array_value
    )
    size = list(
        node.get_parameter("table_size").get_parameter_value().double_array_value
    )

    node.get_logger().info(
        f"Adding table collision box: position={position}, size={size}"
    )
    moveit2.add_collision_box(
        id="table",
        position=position,
        quat_xyzw=[0.0, 0.0, 0.0, 1.0],
        size=size,
    )
    node.get_logger().info("Table collision object added to planning scene.")

    rclpy.shutdown()
    executor_thread.join()
    exit(0)


if __name__ == "__main__":
    main()
```

Deliverable 3: RViz Collision Screenshot

Screenshot folder:

```
/home/ubuntu/techin517/assignments/lab4screenshot
```

Screenshot:



In RViz, I launched MoveIt, started the planning scene node, and moved the interactive planning marker so the arm intersected the table collision object. The robot state turned red / planning failed, showing that the planning scene was active and the table collision was being checked.

Deliverable 4: Save Pose Service

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_functions/soa_functions/save_pose.py
```

This service uses tf2_ros to look up the current transform from base_link to gripper_link. It saves the current end-effector pose to CSV using the columns x,y,z,qx,qy,qz,qw.

```
#!/usr/bin/env python3
"""Save end-effector pose service node."""

import csv
import os

import rclpy
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.duration import Duration
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node
from rclpy.time import Time

from geometry_msgs.msg import Pose
from soa_interfaces.srv import SavePose
import tf2_ros


class SavePoseNode(Node):

    def __init__(self):
        super().__init__('save_pose')

        self._cb_group = ReentrantCallbackGroup()
        self._tf_buffer = tf2_ros.Buffer()
        self._tf_listener = tf2_ros.TransformListener(self._tf_buffer, self)

        self._save_pose_srv = self.create_service(
            SavePose,
            '/follower/save_pose',
            self._handle_save_pose,
            callback_group=self._cb_group,
        )

        self.get_logger().info('SavePose service ready.')

    def _handle_save_pose(self, req, res):
        try:
            transform = self._tf_buffer.lookup_transform(
                'base_link',
                'gripper_link',
                Time(),
                timeout=Duration(seconds=1.0),
            )
        except Exception as e:
            self.get_logger().warn(f'Could not lookup gripper pose: {e}')
            res.success = False
            return res

        pose = Pose()
        pose.position.x = transform.transform.translation.x
        pose.position.y = transform.transform.translation.y
        pose.position.z = transform.transform.translation.z
        pose.orientation = transform.transform.rotation

        res.pose = pose
        res.success = True

        if req.csv_path:
            try:
                self._append_to_csv(req.csv_path, pose)
            except OSError as e:
                self.get_logger().error(f'Failed to write pose CSV: {e}')
                res.success = False

        return res

    def _append_to_csv(self, path: str, pose: Pose) -> None:
        parent_dir = os.path.dirname(path)
        if parent_dir:
            os.makedirs(parent_dir, exist_ok=True)

        has_header = os.path.exists(path) and os.path.getsize(path) > 0

        with open(path, 'a', newline='') as csv_file:
            writer = csv.writer(csv_file)
            if not has_header:
                writer.writerow(['x', 'y', 'z', 'qx', 'qy', 'qz', 'qw'])
```

Deliverable 5: Go To Poses App

Path:

```
/home/ubuntu/techin517/ros2_ws/src/soa_ros2/soa_apps/soa_apps/go_to_poses.py
```

This app loads end-effector poses from a CSV file and sends them to the move_to_pose action server. Since the pose CSV does not contain gripper state, the app sends gripper commands separately through the gripper_command action server.

```
#!/usr/bin/env python3
"""Move through saved poses and gripper commands."""

import csv

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from geometry_msgs.msg import Pose
from soa_interfaces.action import Gripper, MoveToPose


DEFAULT_CSV_PATH = '/home/ubuntu/techin517/ros2_ws/poses.csv'

GRIPPER_OPEN = 1.7453
GRIPPER_CLOSED = 0.1

SEQUENCE = [
    ('pose', 0),
    ('pose', 1),
    ('gripper', GRIPPER_OPEN),
    ('pose', 2),
    ('gripper', GRIPPER_CLOSED),
    ('pose', 3),
]


def load_poses(path: str) -> list:
    poses = []
    with open(path, newline='') as csv_file:
        for row in csv.DictReader(csv_file):
            pose = Pose()
            pose.position.x = float(row['x'])
            pose.position.y = float(row['y'])
            pose.position.z = float(row['z'])
            pose.orientation.x = float(row['qx'])
            pose.orientation.y = float(row['qy'])
            pose.orientation.z = float(row['qz'])
            pose.orientation.w = float(row['qw'])
            poses.append(pose)
    return poses


class GoToPoses(Node):

    def __init__(self):
        super().__init__('go_to_poses')

        self.declare_parameter('csv_path', DEFAULT_CSV_PATH)

        self._pose_client = ActionClient(
            self,
            MoveToPose,
            'move_to_pose',
        )
        self._gripper_client = ActionClient(
            self,
            Gripper,
            'gripper_command',
        )

    def send_pose_goal(self, pose: Pose) -> bool:
        goal = MoveToPose.Goal()
        goal.target_pose = pose

        self.get_logger().info(
            'Sending pose goal: '
            f'x={pose.position.x:.4f}, y={pose.position.y:.4f}, z={pose.position.z:.4f}'
        )

        self._pose_client.wait_for_server()
        future = self._pose_client.send_goal_async(
            goal,
            feedback_callback=self._pose_feedback_callback,
        )
        rclpy.spin_until_future_complete(self, future)
```

App Entry Point Updates

soa_functions/setup.py:

```
from setuptools import find_packages, setup

package_name = 'soa_functions'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ubuntu',
    maintainer_email='42076119+htchr@users.noreply.github.com',
    description='SOA robot function nodes for motion planning and services',
    license='Apache-2.0',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'move_to_pose_server = soa_functions.move_to_pose_server:main',
            'move_to_joint_states_server = soa_functions.move_to_joint_states_server:main',
            'gripper_server = soa_functions.gripper_server:main',
            'controller_switcher = soa_functions.controller_switcher:main',
            'save_joint_states = soa_functions.save_joint_states:main',
            'save_pose = soa_functions.save_pose:main',
            'planning_scene = soa_functions.planning_scene:main',
        ],
    },
)
```

soa_apps/setup.py:

```
from setuptools import find_packages, setup

package_name = 'soa_apps'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ubuntu',
    maintainer_email='42076119+htchr@users.noreply.github.com',
    description='SOA robot application nodes',
    license='Apache-2.0',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
            'go_to_joint_states = soa_apps.go_to_joint_states:main',
            'go_to_poses = soa_apps.go_to_poses:main',
        ],
    },
)
```

Final Run Commands

The robot was run with the following commands in separate terminals.

```
cd /home/ubuntu/techin517/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch soa_moveit_config soa_moveit_bringup.launch.py cameras:=false
```

```
cd /home/ubuntu/techin517/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run soa_functions move_to_pose_server
```

```
cd /home/ubuntu/techin517/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run soa_functions gripper_server
```

```
cd /home/ubuntu/techin517/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run soa_apps go_to_poses --ros-args -p csv_path:=/home/ubuntu/techin517/ros2_ws/poses.csv
```

The final run completed successfully:

```
Pose goal succeeded: Reached target: full pose (tight)
Gripper goal succeeded: Gripper moved to position 1.7453
Pose goal succeeded: Reached target: full pose (relaxed orientation)
Gripper goal succeeded: Gripper moved to position 0.1000
Pose goal succeeded: Reached target: full pose (tight)
=== go_to_poses sequence complete ===
```

Deliverable 6: Video

Video link:

https://drive.google.com/file/d/1Ql06brp-7K1dug9VcE2OKGl-WytyYkhv/view?usp=sharing

The video shows the SO101 arm running the go_to_poses app and lifting the ArUco cube.

Deliverable 7: Forward Kinematics vs Inverse Kinematics

The lab instructions say: "Do not use AI" for this paragraph. □□ must replace this section with an original student-written paragraph before submitting.

Student-written paragraph:

Forward kinematics and inverse kinematics are used for different purposes in robotics. Forward kinematics is used when we already know the joint angles of a robot and want to calculate the position and orientation of the end effector. For example, if a robotic arm has specific joint values, forward kinematics tells us where the gripper is in space. I would use forward kinematics when checking or simulating the current pose of a robot. Inverse kinematics is used in the opposite situation: when we know the desired position of the end effector and need to find the joint angles that can reach that position. For example, if I want a robot arm to pick up an object at a certain location, I would use inverse kinematics to compute how each joint should move. In general, forward kinematics answers “where is the robot now?” while inverse kinematics answers “how should the robot move to reach a target?”

Build And Verification

The Lab 4 packages were built and verified with:

```
cd /home/ubuntu/techin517/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --packages-up-to soa_moveit_config
colcon build --build-base /tmp/lab4_build_actual2 --packages-select soa_interfaces soa_functions soa_apps
source install/setup.bash
ros2 interface show soa_interfaces/action/MoveToPose
ros2 pkg executables soa_functions
ros2 pkg executables soa_apps
```

The following executables were available:

```
soa_functions move_to_pose_server
soa_functions planning_scene
soa_functions save_pose
soa_apps go_to_poses
```